Plan de Game Day

Actividad 3.2 - Observabilidad en Ambientes Productivos
Maestría en Arquitectura de Software | Universidad de La Sabana
18 de octubre de 2026 (Colombia)

Entregado por: Manuel Fernando Santofimio Tovar y Beycy Yuliet Rojas

1. Objetivo y arquitectura objetivo

Evaluar experimentalmente la resiliencia del microservicio FastAPI del laboratorio anterior ante degradación de red, agotamiento de CPU y pérdida de una réplica. Se parte de una salida observable, se introduce una perturbación acotada y se compara el comportamiento con una hipótesis previa. El propósito SRE es convertir la debilidad descubierta en una mejora verificable; causar fallos por sí solo no demuestra resiliencia.

Objetivo identificable: repositorio yuliet719-beep/mi-microservicio, código app/app.py del commit 7d4e7f1913f2. Endpoints GET /hello (respuesta funcional), /health (salud del proceso) y /metrics (instrumentación Prometheus). No tiene base de datos, colas ni llamadas a servicios externos. Sus dependencias de ejecución son FastAPI, Uvicorn, Python y prometheus-fastapi-instrumentator.

Capa	Arquitectura anterior y alcance del ensayo
Entrega y despliegue	GitHub Actions → Jenkins → registro Docker → Helm/Argo CD → Kubernetes. Son componentes de entrega; no se perturban ni se afirma haberlos ejecutado aquí.
Ejecución local	Cliente Python en Windows → puerto 127.0.0.1:18082 → target:8000 (una réplica de FastAPI). Docker Compose crea una red exclusiva.
Observabilidad	Prometheus → target:8000/metrics cada 2 s; API en 127.0.0.1:19092. El repositorio anterior dispone además de Grafana y alertas. El ensayo conserva registros y métricas sin depender de Grafana.

Adaptación explícita: la imagen original es distroless y no contiene tc. La imagen sandbox conserva el mismo código y requisitos, usa python:3.11-slim y añade iproute2. Solo target recibe NET_ADMIN, 1 CPU y 256 MiB. Los puertos se enlazan a loopback. No se usa red del host ni un clúster productivo. Estas diferencias limitan la extrapolación a Kubernetes.

2. Preparación del Game Day

Estado estable común: tráfico sintético GET /hello a 5 solicitudes/s, sin reintentos, timeout cliente de 2 s, disponibilidad HTTP ≥99% y p95 extremo a extremo <200 ms en referencia de 30 s. Se usan percentiles de rango más cercano e incluyen fallos/timeouts. Ventanas cortas sirven al laboratorio; no equivalen a un SLO mensual.

Roles operativos: director autoriza inicio/parada y mantiene el alcance; operador inyecta y revierte; observador registra tiempos y valida SLI. Son responsabilidades del plan, no una atribución de participación individual. Antes de inyectar: verificar /health, recolección UP, puertos libres, estado estable y comando de retirada. Parar ante diez errores consecutivos o pérdida de control del inyector.

3. Hipótesis previas y experimentos alineados

H1. En estado estable, cuando se agregan 300 ms de latencia de salida a la única réplica sandbox durante un máximo de 60 s, esperamos que la disponibilidad HTTP sea ≥99%, el p95 extremo a extremo permanezca <200 ms y, tras el rollback, vuelva a ese umbral en 30 s. Se busca refutar la conservación del estado estable: la ausencia de tolerancia al retardo es una debilidad comprobable. Hipótesis y límites quedaron guardados antes de inyectar en plan_previo.json.

H2. En estado estable, cuando un proceso de estrés consume la cuota de una CPU del contenedor sandbox durante 60 s, esperamos disponibilidad ≥99%, p95 <200 ms y recuperación a los umbrales en 30 s. La hipótesis evalúa si el margen de capacidad de una sola réplica evita degradación bajo competencia de CPU.

H3. En estado estable, cuando se pierde la única réplica durante 20 s, esperamos disponibilidad ≥99% durante la perturbación y recuperación de respuestas 2xx en ≤30 s desde la restauración. Es una hipótesis de continuidad intencionalmente exigente para revelar el punto único de fallo. No se presupone que Kubernetes o un balanceador salven una réplica inexistente en Compose.

Diseño	Fallo, radio y duración	Inyección y recuperación automática
E1 / H1 EJECUTADO	Latencia de red. Solo eth0 de target; afecta respuestas /hello, salud y scrape de esa réplica. Máximo 60 s: suficiente para 30 scrapes y 300 solicitudes previstas.	tc qdisc replace dev eth0 root netem delay 300ms. Watchdog en contenedor: sleep 60; tc qdisc del dev eth0 root. El cliente retira qdisc también en finally y ante parada.
E2 / H2 DISEÑADO	Resource exhaustion (CPU). Solo target con cuota 1 CPU. 60 s para observar contención sin prolongar presión. Prometheus se mantiene fuera de esa cuota.	Preparar stress-ng desde Debian y GNU timeout en sandbox. Ejecutar timeout --signal=TERM --kill-after=5s 60s stress-ng --cpu 2 --timeout 60s. El timeout propio termina la carga a 60 s; el supervisor fuerza fin en 65 s. Verificar fin de trabajadores.
E3 / H3 DISEÑADO	Error/caída de proceso: indisponibilidad de una réplica. Solo target, 20 s. No se presenta como partición de red.	Armar proceso watchdog independiente en el host: sleep 20 s y compose start target; luego compose kill -s SIGKILL target. Finally vuelve a iniciar target. Verificar salud y UP. Código concreto en README; solo afecta target, sin eliminar datos.

Seguridad y equivalencia conceptual: en Gremlin se modelarían ataques de latencia y CPU sobre un único contenedor; en Litmus, un ChaosEngine seleccionaría una etiqueta sandbox con duración y sondas de salud, usando pod-delete para E3. Chaos Monkey representa la pérdida de instancia de E3. Son mapeos de diseño, no herramientas desplegadas. El mecanismo ejecutado es tc netem, solicitado por la guía.

Secuencia: briefing y preflight → 30 s de referencia → inyección → máximo 60 s (E1/E2) o 20 s (E3) → retirada → 30 s de recuperación → comparación y retrospectiva. Ejecutar un fallo a la vez; no iniciar E2/E3 si E1 deja degradación residual. El director puede detener antes del límite con el rollback documentado.

4. Observabilidad y criterios de validación

SLI y cálculo	Criterio y evidencia
Disponibilidad funcional	Solicitudes 2xx / solicitudes totales; timeout cuenta como fallo. Umbral ≥99% en cada fase. CSV: status y error. No confundir up=1 (scrape exitoso) con disponibilidad del usuario.
Latencia de usuario	p95 de duration_ms por fase, incluyendo tiempo consumido por fallos; objetivo <200 ms. También proporción de respuestas 2xx en <200 ms. CSV permite recalcular ambos.
Tasa de error y throughput	1 - disponibilidad; éxitos / duración real de fase en segundos. Carga ofrecida: 5/s con hasta 12 trabajadores, timeout 2 s y sin reintentos. Si aparece cola, throughput obtenido puede ser menor.
Recuperación	Comprobar qdisc noqueue y la ventana posterior de 30 s: disponibilidad ≥99%, p95 <200 ms. Esto confirma recuperación dentro de la ventana; no mide un MTTR exacto con resolución de milisegundos.
CPU y memoria (H2)	rate(process_cpu_seconds_total[30s]) y process_resident_memory_bytes. Son señales de contexto; CPU del proceso no incluye necesariamente al proceso stress-ng. Usar docker stats del contenedor para verificar saturación total.

Consultas propuestas: sum(rate(http_requests_total{handler="/hello",status="2xx"}[30s])) / sum(rate(http_requests_total{handler="/hello"}[30s])) para éxito servidor; sum(rate(http_requests_total{handler="/hello"}[30s])) para tráfico; histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket{handler="/hello"}[30s]))) para p95 del handler. Confirmar etiquetas en /metrics antes de reutilizar. La disponibilidad cliente sigue siendo autoritativa cuando las solicitudes no llegan al servidor.

En E1 se deben contrastar latencia cliente y duración interna del handler: netem actúa fuera del código Python y puede degradar la red mientras /health y el handler parecen sanos. En E2 se espera aumento de latencia y CPU antes de errores. En E3 se esperan errores de conexión, caída de up y registros de parada/arranque. Las tres hipótesis se rechazan si incumplen cualquiera de sus objetivos, aunque el rollback funcione.

Registros, spans y límites de correlación

solicitudes.csv registra UTC, fase, duración, estado, error y trace_id único enviado como X-Request-ID. Son spans mínimos del cliente de ensayo, no trazas distribuidas. servidor.log contiene accesos y ciclo de vida de Uvicorn. eventos.json relaciona tiempos de inyección/retirada, comandos, códigos de retorno y salida de tc. Los snapshots de /metrics y las consultas de up son evidencia adicional del proceso y de la recolección.

El servicio original no registra ni propaga X-Request-ID y carece de OpenTelemetry: no hay enlace probado entre un span cliente y una línea de Uvicorn. La correlación actual es por ventana UTC y fase. Como remediación, instrumentar FastAPI con spans servidor, propagar traceparent y registrar trace_id/span_id en JSON; entonces comparar spans cliente/red/servidor para localizar el retardo.

Condiciones de parada y límites

Se aborta si la referencia no cumple o se observan diez fallos consecutivos. El límite independiente de 60 s evita que la salida del cliente deje netem permanente; finally retira anticipadamente el fallo y la verificación qdisc valida el efecto. Las llamadas fallidas conservan su duración y error. No se eliminan pruebas que contradigan la hipótesis. El README concreta el supervisor E2 y el watchdog E3; se deben validar antes de ejecutarlos. E2/E3 siguen siendo diseños, no resultados medidos.

5. Ejecución real y evidencia

Inicio UTC: 2026-10-19T03:05:19.509572+00:00; fin UTC: 2026-10-19T03:06:30.509658+00:00. Corresponde al 18 de octubre por la noche en Colombia (UTC-5). Se ejecutó E1 en Docker Desktop con Linux, FastAPI original y Prometheus. E2/E3 permanecen como diseños, cumpliendo el mínimo de un experimento ejecutado.

Procedimiento reproducible: (1) construir e iniciar compose.yaml; (2) ejecutar python "Actividad 3.2 - Laboratorio Plan de Game Day/ejecutar.py"; (3) medir referencia; (4) programar watchdog e instalar netem; (5) muestrear hasta límite o parada; (6) retirar en finally, comprobar qdisc y medir recuperación. Comandos completos y salidas están en eventos.json. El script calcula automáticamente resultados.json desde las muestras; no introduce valores esperados como resultados.

Fase	Solicitudes / éxito	Disponibilidad	p95 cliente	2xx <200 ms
Referencia	150 / 150	100.00%	26.5 ms	100.0%
Con fallo	36 / 15	41.67%	2038.1 ms	0.0%
Recuperación	150 / 150	100.00%	26.2 ms	99.3%

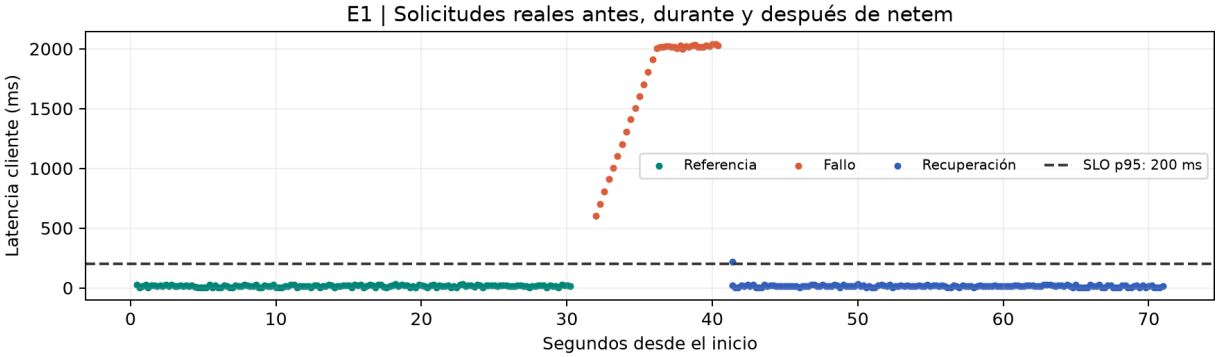


Figura 1. Gráfico derivado de solicitudes.csv. Cada punto es una solicitud real; las duraciones fallidas incluyen el tiempo de espera. La línea indica el umbral de latencia definido previamente.

Esperado frente a obtenido: H1 exigía ≥99% de éxito y p95 <200 ms durante la perturbación. Se obtuvieron 41.67% y 2038.1 ms, con throughput de 1.66 éxitos/s. La fase duró 9.03 s y activó parada anticipada por diez fallos consecutivos. H1 queda **refutada**. No se afirma exposición completa de 60 s: ese valor era el máximo de seguridad.

Recuperación: después de eliminar qdisc, la ventana de recuperación obtuvo 100.00% de disponibilidad y p95 de 26.2 ms. La reversibilidad se valida con la salida noqueue de tc y las respuestas posteriores. El watchdog es una segunda barrera; la retirada anticipada efectiva la realizó finally.

El preensayo también abortó; se conserva en preensayo_abortado. Su cliente inicial no persistía muestras de la fase interrumpida, por lo que se corrigió la persistencia y se repitió la prueba. El CSV principal sí contiene los fallos y la recuperación. Esta distinción evita presentar la referencia del preensayo como un ensayo completo.

6. Debilidad sistémica y remediación

La única réplica no conservó la experiencia de usuario al degradar su enlace. Los errores de conexión/espera del CSV y la recuperación al retirar netem muestran vulnerabilidad al fallo inyectado. No permiten atribuir todo el retardo a los 300 ms nominales: handshake TCP, reintentos de transporte, planificación y traducción de red de Docker pueden amplificarlo. Para aislar esa causalidad se necesitarían captura TCP y otro cliente dentro de la red; aquí se reporta el efecto extremo a extremo observado.

La salud /health solo indica que el proceso responde cuando se logra acceder; no demuestra cumplimiento del SLO. Las métricas del handler no incluyen toda la red. La alerta anterior MicroserviceUnavailable exige un minuto y HighProcessMemory cinco minutos, por lo que no aseguran detección oportuna de una degradación de latencia breve. Tener recolección UP tampoco demuestra que el usuario reciba respuestas rápidas.

Prioridad / acción	Validación de cierre y responsable operativo
P1 - SLI externo y alerta de latencia	Observador: añadir sondeo externo y alerta sobre proporción de solicitudes rápidas/errores. Repetir E1 y medir tiempo de detección ≤10 s en sandbox; separar alerta del handler y del camino de red.
P1 - Eliminar punto único de fallo	Operador: dos réplicas detrás de un balanceador con readiness real. Repetir E3 sobre una réplica; exigir ≥99% de éxito y documentar tiempo de exclusión/reintegración. No asumir que duplicar réplicas corrige una red compartida.
P2 - Presupuesto de latencia	Operador: definir timeout de conexión/lectura coherente con SLO; limitar concurrencia. Si se añade dependencia, usar circuit breaker y reintentos limitados con backoff/jitter solo en operaciones idempotentes. Repetir E1 y comprobar que no hay tormenta de reintentos.
P2 - Capacidad y trazabilidad	Observador: perfilar CPU, ajustar requests/limits y evaluar autoscaling con H2. Agregar OpenTelemetry y logs estructurados; validar propagación de trace_id entre cliente y servidor.

7. Aprendizaje SRE y siguiente Game Day

La prueba aporta evidencia de una hipótesis falsa y de un rollback efectivo. La mejora antifrágil consiste en cerrar las acciones con responsables, repetir el mismo plan y comparar resultados, conservando ambos ensayos. El director revisará los umbrales antes de cada repetición; solo ampliar radio y duración después de demostrar detección, recuperación y continuidad. E2/E3 deben ensayarse después en sandbox con las mismas mediciones. No se certifica resiliencia productiva a partir de una muestra local corta.

8. Índice de evidencia y fuentes

evidencias/resultados.json: resumen y procedencia; solicitudes.csv: datos por petición; eventos.json: tc, tiempos y rollback; metrics_baseline.txt y metrics_recovery.txt: instrumentación; prometheus_*.json: recolección; servidor.log: proceso; prometheus_captura.jpg: captura de Targets; resultados.png: figura calculada. README.md contiene ejecución, rollback y auditoría individual de los 20 criterios. criterios_fuente.txt conserva las dos listas originales de evaluación. La captura muestra recolección disponible, no prueba por sí sola H1.

Fuentes consultadas el 18/10/2026 (Colombia):
[1] Consigna y rúbrica de Actividad 3.2, Universidad de La Sabana: https://unisabanae-learning.unisabana.edu.co/mod/assign/view.php?id=306420
[2] Principles of Chaos Engineering: https://principlesofchaos.org/
[3] Manual tc-netem(8): https://man7.org/linux/man-pages/man8/tc-netem.8.html
[4] Prometheus, Histograms and summaries: https://prometheus.io/docs/practices/histograms/
[5] Código base: https://github.com/yuliet719-beep/mi-microservicio/tree/7d4e7f1913f26262ff23f9deee14ebbfe2b3f797